

ECS Project Step 2.1 – Create Docker Image from Code

To Begin with the Lab

Summary of the Lab

In this step, a Docker build environment was created on an EC2 instance, the PHP application was cloned from the CloudFolks public GitHub repository, Composer and PHP dependencies were installed, and a Docker image was built from the provided Dockerfile. The instructor also explained why Composer is required: the application uses the AWS SDK for PHP to interact with Amazon S3, and that SDK is installed through Composer. The result of this step is a ready-to-use Docker image that can later be pushed to Amazon ECR and used by ECS tasks or services.

Creating the Docker Image

The purpose of this step is to prepare the container image for the project application. The instructor makes it clear that ECS cannot build images; it only runs them. So before the application can be deployed to ECS, the source code must be turned into a Docker image on a separate machine. In this project, that machine is a Docker-enabled EC2 instance called the **build machine**.

Lab Steps

Step 1 – Launch the Build Machine

- Open the **Amazon EC2 Console**.
- Click **Launch instance**.

Resources

You are using the following Amazon EC2 resources in the United States (N. Virginia) Region:

Instances (running)	1	Auto Scaling Groups	1	Capacity Reservations	0
Dedicated Hosts	0	Elastic IPs	0	Instances	2
Key pairs	31	Load balancers	0	Placement groups	0
Security groups	35	Snapshots	0	Volumes	1

Launch instance

To get started, launch an Amazon EC2 instance, which is a virtual server in the cloud.

Launch instance [Migrate a server](#)

Note: Your instances will launch in the United States (N. Virginia) Region

Instance alarms [View in CloudWatch](#)

0 in alarm 0 OK 0 insufficient data

Instances in alarm

Service health [AWS Health Dashboard](#)

Region: United States (N. Virginia)

Status: This service is operating normally.

Zones

Zone name	Zone ID
us-east-1a	use1-az4
us-east-1b	use1-az6
us-east-1c	use1-az1

EC2 cost

Date range: Past 6 months | Region: Global

Total cost

Regions

Analyze your costs in Cost Explorer

Account attributes

Default VPC: vpc-099792461c7d0a616

Settings: Data protection and security, Allowed AMIs, Zones, EC2 Serial Console, Default credit specification, Managed resources, EC2 console preferences

- Enter the instance name: Build-Machine
- Select Amazon Linux 2023.

Name and tags

Name: Build-Machine

Application and OS Images (Amazon Machine Image)

Search our full catalog including 1000s of application and OS images

Recents | My AMIs | Quick Start

Amazon Linux | macOS | Ubuntu | Windows | Red Hat | SUSE Linux | Debian

Amazon Machine Image (AMI)

Amazon Linux 2023 kernel-6.18 AMI

Free tier eligible

Summary

Number of instances: 1

Software Image (AMI): Amazon Linux 2023 AMI 2023.12...read more

Virtual server type (instance type): t3.micro

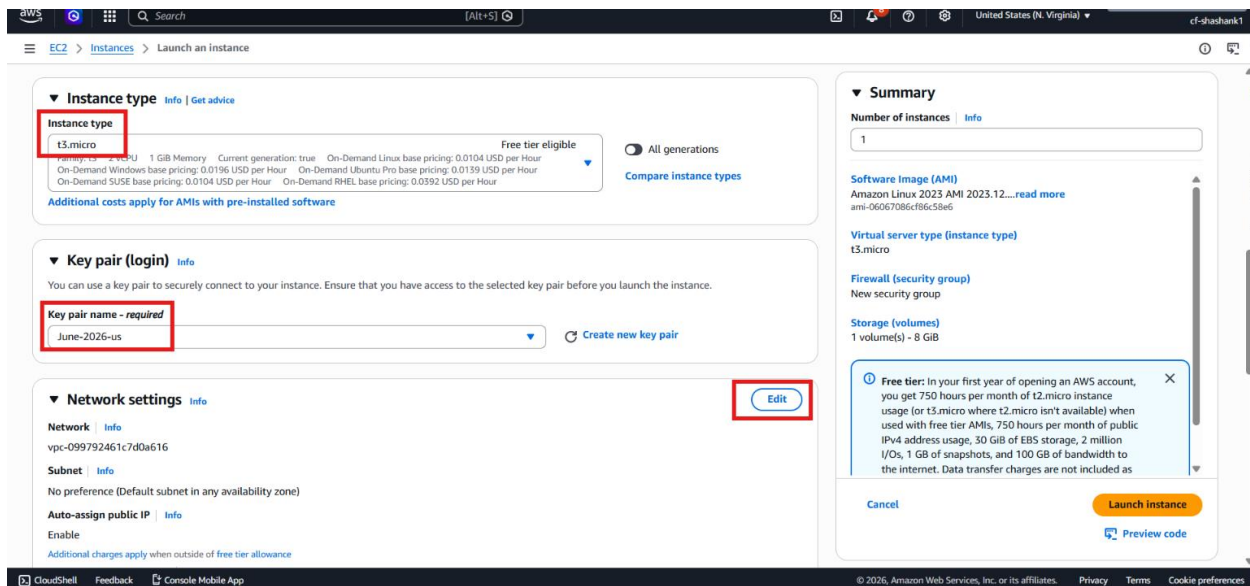
Firewall (security group): New security group

Storage (volumes): 1 volume(s) - 8 GiB

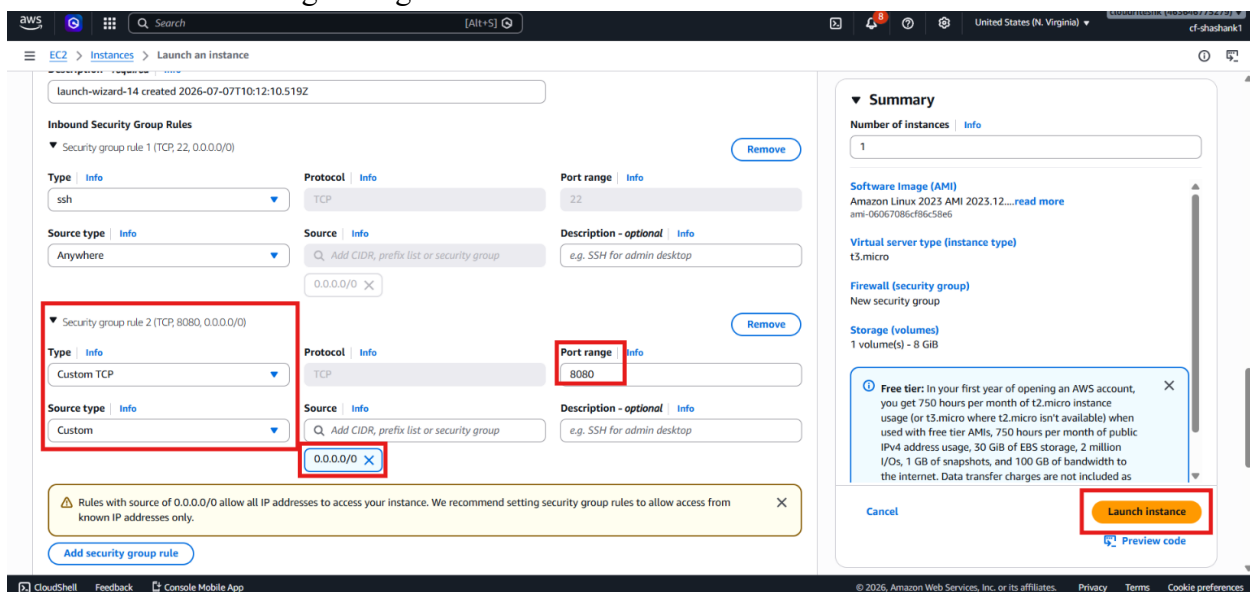
Free tier: In your first year of opening an AWS account, you get 750 hours per month of t2.micro instance usage (or t3.micro where t2.micro isn't available) when used with free tier AMIs, 750 hours per month of public IPv4 address usage, 30 GiB of EBS storage, 2 million I/Os, 1 GiB of snapshots, and 100 GiB of bandwidth to the internet. Data transfer charges are not included as

Cancel [Launch instance](#) [Preview code](#)

- Choose t3.micro.
- Select or create a key pair.



- Click **Edit** and Open inbound ports:
 - SSH 22
 - HTTP 8080
- Leave the remaining settings as default.



- Launch the instance.

Step 2 – Connect to the EC2 Instance

- Copy the public IP of the build machine.
- Connect using SSH:

```
ssh -i "your-key.pem" ec2-user@<your-ec2-public-ip>
```

- Confirm that you can access the instance as ec2-user.

Step 4 – Install PHP, Git, and Composer

- Install PHP CLI, unzip, and git:

```
sudo dnf install php-cli unzip git -y
```

- Download the official Composer installer:

```
php -r "copy('https://getcomposer.org/installer', 'composer-setup.php');"
```

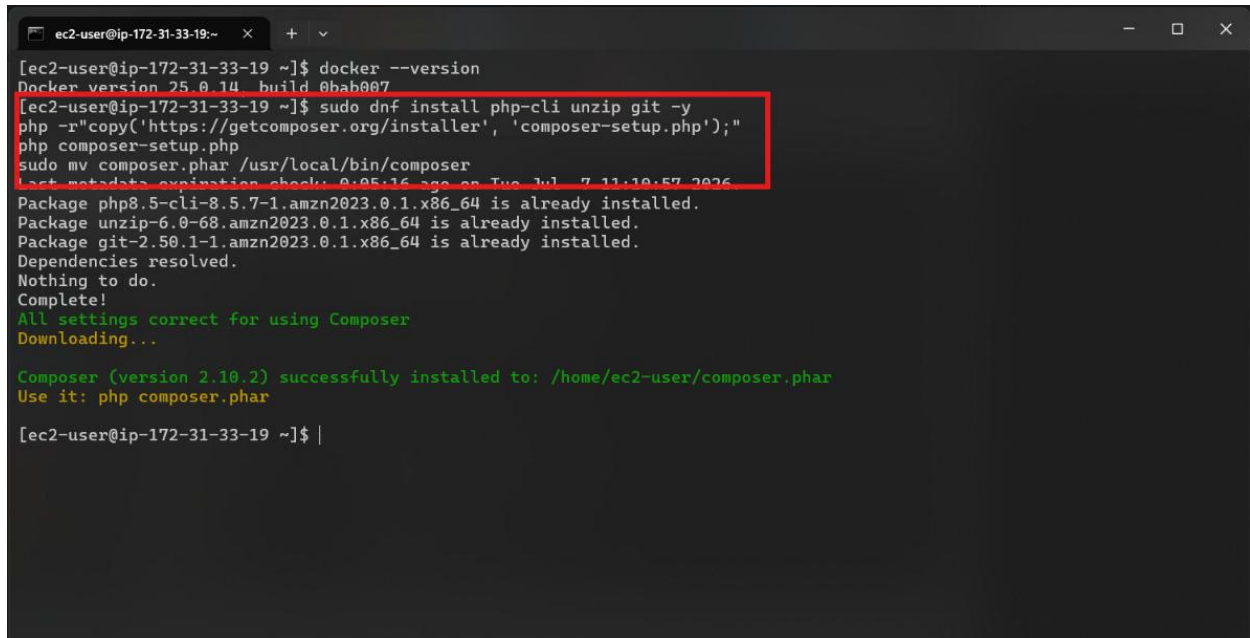
- Run the Composer installer:

```
php composer-setup.php
```

- Move Composer to a global location:

```
sudo mv composer.phar /usr/local/bin/composer
```

Composer is needed because the application uses the **AWS SDK for PHP**, which is a third-party library and not built into PHP itself.



```
ec2-user@ip-172-31-33-19:~$ docker --version
Docker version 25.0.14 build 0bab007
ec2-user@ip-172-31-33-19:~$ sudo dnf install php-cli unzip git -y
php -r"copy('https://getcomposer.org/installer', 'composer-setup.php');"
php composer-setup.php
sudo mv composer.phar /usr/local/bin/composer
Last metadata expiration check: 0:05:16 ago on Tue Jul 7 11:10:57 2025.
Package php8.5-cli-8.5.7-1.amzn2023.0.1.x86_64 is already installed.
Package unzip-6.0-68.amzn2023.0.1.x86_64 is already installed.
Package git-2.50.1-1.amzn2023.0.1.x86_64 is already installed.
Dependencies resolved.
Nothing to do.
Complete!
All settings correct for using Composer
Downloading...

Composer (version 2.10.2) successfully installed to: /home/ec2-user/composer.phar
Use it: php composer.phar

ec2-user@ip-172-31-33-19:~$ |
```

Step 5 – Clone the Application Repository

- Clone the CloudFolks public PHP app repository:

```
git clone https://github.com/ShashankRajput90/CloudFreeks-S3-PHP.git
```

- Change into the application directory:

```
cd CloudFreeks-S3-PHP
```

This repository contains the application source code, the Dockerfile, and the dependency files.

```
ec2-user@ip-172-31-33-19:~/ / X + v
Docker version 25.0.14, build 0bab007
[ec2-user@ip-172-31-33-19 ~]$ sudo dnf install php-cli unzip git -y
php -r"copy('https://getcomposer.org/installer', 'composer-setup.php');"
php composer-setup.php
sudo mv composer.phar /usr/local/bin/composer
Last metadata expiration check: 0:05:16 ago on Tue Jul 7 11:10:57 2026.
Package php8.5-cli-8.5.7-1.amzn2023.0.1.x86_64 is already installed.
Package unzip-6.0-68.amzn2023.0.1.x86_64 is already installed.
Package git-2.50.1-1.amzn2023.0.1.x86_64 is already installed.
Dependencies resolved.
Nothing to do.
Complete!
All settings correct for using Composer
Downloading...

Composer (version 2.10.2) successfully installed to: /home/ec2-user/composer.phar
Use it: php composer.phar

[ec2-user@ip-172-31-33-19 ~]$ git clone https://github.com/ShashankRajput90/CloudFreeks-S3-PHP.git
Cloning into 'CloudFreeks-S3-PHP'...
remote: Enumerating objects: 16, done.
remote: Counting objects: 100% (16/16), done.
remote: Compressing objects: 100% (13/13), done.
remote: Total 16 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (16/16), 27.51 KiB | 27.51 MiB/s, done.
Resolving deltas: 100% (1/1), done.
[ec2-user@ip-172-31-33-19 ~]$ ls
CloudFreeks-S3-PHP composer-setup.php
[ec2-user@ip-172-31-33-19 ~]$ cd CloudFreeks-S3-PHP
[ec2-user@ip-172-31-33-19 CloudFreeks-S3-PHP]$
```

Step 6 – Install the Application Dependencies

- Install the PHP XML extension:

```
sudo dnf install php-simplexml -y
```

This is needed because AWS SDK uses XML responses internally for some AWS service communications.

```
ec2-user@ip-172-31-33-19:~/ / X + v
[ec2-user@ip-172-31-33-19 ~]$ ls
CloudFreeks-S3-PHP composer-setup.php
[ec2-user@ip-172-31-33-19 ~]$ cd CloudFreeks-S3-PHP
[ec2-user@ip-172-31-33-19 CloudFreeks-S3-PHP]$ sudo dnf install php-simplexml -y
Last metadata expiration check: 0:14:30 ago on Tue Jul 7 11:10:57 2026.
Dependencies resolved.
=====
Package                Architecture    Version                               Repository          Size
=====
Installing:
php8.5-xml              x86_64         8.5.7-1.amzn2023.0.1                amazonlinux          248 k
Installing dependencies:
libxslt                x86_64         1.1.43-1.amzn2023.0.3                amazonlinux          183 k

Transaction Summary
=====
Install 2 Packages

Total download size: 431 k
Installed size: 1.2 M
Downloading Packages:
(1/2): libxslt-1.1.43-1.amzn2023.0.3.x86_64.rpm                4.5 MB/s | 183 kB    00:00
(2/2): php8.5-xml-8.5.7-1.amzn2023.0.1.x86_64.rpm              5.2 MB/s | 248 kB    00:00
-----
Total                                                                4.3 MB/s | 431 kB    00:00
Running transaction check
Transaction check succeeded.
Running transaction test
Transaction test succeeded.
Running transaction
```

- Run Composer install:

```
composer install
```


This installs the PHP libraries defined in the project dependencies. The instructor explained that the developer usually provides these files, and the build process installs them based on the `composer.json` and `lock` files.

```
ec2-user@ip-172-31-33-19:~/ / X + v
Completed!
[ec2-user@ip-172-31-33-19 CloudFreeeks-S3-PHP]$ composer install
Installing dependencies from lock file (including require-dev)
Verifying lock file contents can be installed on current platform.
Package operations: 12 installs, 0 updates, 0 removals
- Downloading psr/http-message (2.0)
- Downloading symfony/polyfill-mbstring (v1.32.0)
- Downloading mtdowling/jmespath.php (2.8.0)
- Downloading ralouphie/getallheaders (3.0.3)
- Downloading psr/http-factory (1.1.0)
- Downloading guzzlehttp/psr7 (2.7.1)
- Downloading guzzlehttp/promises (2.2.0)
- Downloading symfony/deprecation-contracts (v3.6.0)
- Downloading psr/http-client (1.0.3)
- Downloading guzzlehttp/guzzle (7.9.3)
- Downloading aws/aws-crt-php (v1.2.7)
- Downloading aws/aws-sdk-php (3.347.0)
- Installing psr/http-message (2.0): Extracting archive
- Installing symfony/polyfill-mbstring (v1.32.0): Extracting archive
- Installing mtdowling/jmespath.php (2.8.0): Extracting archive
- Installing ralouphie/getallheaders (3.0.3): Extracting archive
- Installing psr/http-factory (1.1.0): Extracting archive
- Installing guzzlehttp/psr7 (2.7.1): Extracting archive
- Installing guzzlehttp/promises (2.2.0): Extracting archive
- Installing symfony/deprecation-contracts (v3.6.0): Extracting archive
- Installing psr/http-client (1.0.3): Extracting archive
- Installing guzzlehttp/guzzle (7.9.3): Extracting archive
- Installing aws/aws-crt-php (v1.2.7): Extracting archive
- Installing aws/aws-sdk-php (3.347.0): Extracting archive
Generating autoload files
5 packages you are using are looking for funding.
Use the `composer fund` command to find out more!
[ec2-user@ip-172-31-33-19 CloudFreeeks-S3-PHP]$
```

Step 7 – Build the Docker Image

- Build the image from the current directory:

```
docker build -t cloudfreeks-s3-php .
```

The `.` is important because Docker must find the `Dockerfile` in the current folder.

```
ec2-user@ip-172-31-33-19:~/( X + v
[ec2-user@ip-172-31-33-19:~/( X + v]# docker build -t cloudfreeks-s3-php .
[+] Building 0.0s (0/0)
ERROR: invalid tag "cloudfreeks-s3-php": repository name must be lowercase
[ec2-user@ip-172-31-33-19:~/( X + v]# docker build -t cloudfreeks-s3-php .
[+] Building 2.9s (4/0)
=> [internal] load build definition from Dockerfile
=> transferring dockerfile: 309B
=> [internal] load metadata for docker.io/library/php:8.2-apache
=> [internal] load .dockerignore
=> transferring context: 2B
=> [1/4] FROM docker.io/library/php:8.2-apache@sha256:848bd4a97499465e365eb4681142332a958701cb6d2fff77342f02699b
=> resolve docker.io/library/php:8.2-apache@sha256:848bd4a97499465e365eb4681142332a958701cb6d2fff77342f02699b
=> sha256:848bd4a97499465e365eb4681142332a958701cb6d2fff77342f02699b81985f 10.35kB / 10.35kB
=> sha256:290b798bb8891c866dafd81cebb75735877c5ec60d97b9e0b1a9e0c67cd37a03 3.82kB / 3.82kB
=> sha256:43654fbbfeb12b01906f8be3836816e0a3e3ea6c970f859ff7fa469594e939ee 225B / 225B
=> sha256:97ba1c99696383b6c80dbd5154d095f938d37d73db6b2f4d91d7d0fc62c1ffa 11.92kB / 11.92kB
=> sha256:e95a6c7ea7d49b37928899b023ecd0e32796c976c1748491f76cae53ba86d13a 29.79MB / 29.79MB
=> sha256:a4efc98a7f4ff396aceb7c7431d296b3b75d15d922e504ba8ee58ee0344daa48 117.84MB / 117.84MB
=> sha256:05f27c6ae9385bea512e3e98ee36fd78d6f197b60d9797bf4087832e8cbd8234 224B / 224B
=> sha256:e9571d1330ad361a711f35739ca06a464c392a1f295c37eac881fe826898f4ee 4.23MB / 4.23MB
=> sha256:ddd74371c4514dd83bb24a22a83c96765aa9ad08b096afba58ad27af5276fb18 427B / 427B
=> sha256:d5c4f02e98eb5bcd34bf11d61ec8636eef68ec93817c4cd5cf696d197b7411f 480B / 480B
=> sha256:07c774bd9142872edd39d4d6a3b59d54c780e3923e2521lead0a61ddd9d871e1 12.33MB / 12.33MB
=> extracting sha256:e95a6c7ea7d49b37928899b023ecd0e32796c976c1748491f76cae53ba86d13a
=> sha256:70163b1602d15f979d01e34dc2b40e4abe5c020637faee58e05bf07852e95e 491B / 491B
=> sha256:ccb8dce9264bc150c2780db2aa18d5f7f627e360525435d7b837001e7699e2aa 11.46MB / 11.46MB
=> sha256:035732560bf3d2e6f7e64e06917a2b527aa778837b3447a141be7775958bcca9 2.46kB / 2.46kB
=> sha256:4b1ebd1b6a4197fa3404d6d4f26d2130dbe1833e5fe0dcfce3ab385bb0e892ad 247B / 247B
=> sha256:f3170b4790dd270f8281e1ed417471130aaab302fac01e03d482fe2a3c9346fc 242B / 242B
=> sha256:b206b1362c592cc757b9cb36ba8882e04b897d0645bc0b24773f215c356f7ce2 889B / 889B
=> sha256:4f4fb700ef54461cfa02571ae0db9a0dc1e0c0b5577484a6d75e68dc38e8acc1 32B / 32B
=> [internal] load build context
=> transferring context: 42.58MB
```

Step 8 – Verify the Image

- List the local Docker images

docker images

- Confirm that the image named cloudfreeks-s3-php-app is present.
- If the image appears, the build was successful.

```
ec2-user@ip-172-31-33-19:~/( X + v
=> sha256:70163b1602d15f979d01e34dc2b40e4abe5c020637faee58e05bf07852e95e 491B / 491B
=> sha256:ccb8dce9264bc150c2780db2aa18d5f7f627e360525435d7b837001e7699e2aa 11.46MB / 11.46MB
=> sha256:035732560bf3d2e6f7e64e06917a2b527aa778837b3447a141be7775958bcca9 2.46kB / 2.46kB
=> sha256:4b1ebd1b6a4197fa3404d6d4f26d2130dbe1833e5fe0dcfce3ab385bb0e892ad 247B / 247B
=> sha256:f3170b4790dd270f8281e1ed417471130aaab302fac01e03d482fe2a3c9346fc 242B / 242B
=> sha256:b206b1362c592cc757b9cb36ba8882e04b897d0645bc0b24773f215c356f7ce2 889B / 889B
=> sha256:4f4fb700ef54461cfa02571ae0db9a0dc1e0c0b5577484a6d75e68dc38e8acc1 32B / 32B
=> extracting sha256:43654fbbfeb12b01906f8be3836816e0a3e3ea6c970f859ff7fa469594e939ee
=> extracting sha256:a4efc98a7f4ff396aceb7c7431d296b3b75d15d922e504ba8ee58ee0344daa48
=> extracting sha256:05f27c6ae9385bea512e3e98ee36fd78d6f197b60d9797bf4087832e8cbd8234
=> extracting sha256:e9571d1330ad361a711f35739ca06a464c392a1f295c37eac881fe826898f4ee
=> extracting sha256:ddd74371c4514dd83bb24a22a83c96765aa9ad08b096afba58ad27af5276fb18
=> extracting sha256:d5c4f02e98eb5bcd34bf11d61ec8636eef68ec93817c4cd5cf696d197b7411f
=> extracting sha256:07c774bd9142872edd39d4d6a3b59d54c780e3923e2521lead0a61ddd9d871e1
=> extracting sha256:70163b1602d15f979d01e34dc2b40e4abe5c020637faee58e05bf07852e95e
=> extracting sha256:ccb8dce9264bc150c2780db2aa18d5f7f627e360525435d7b837001e7699e2aa
=> extracting sha256:035732560bf3d2e6f7e64e06917a2b527aa778837b3447a141be7775958bcca9
=> extracting sha256:4b1ebd1b6a4197fa3404d6d4f26d2130dbe1833e5fe0dcfce3ab385bb0e892ad
=> extracting sha256:f3170b4790dd270f8281e1ed417471130aaab302fac01e03d482fe2a3c9346fc
=> extracting sha256:b206b1362c592cc757b9cb36ba8882e04b897d0645bc0b24773f215c356f7ce2
=> extracting sha256:4f4fb700ef54461cfa02571ae0db9a0dc1e0c0b5577484a6d75e68dc38e8acc1
=> [internal] load build context
=> transferring context: 42.58MB
=> [2/4] RUN docker-php-ext-install mysqli
=> [3/4] WORKDIR /var/www/html
=> [4/4] COPY . .
=> exporting to image
=> exporting layers
=> writing image sha256:b31be780b9c5eff3de8f1ae80f305d81f1de2dcdf15fadca5d3e1314420e697d
=> naming to docker.io/library/cloudfreeks-s3-php
[ec2-user@ip-172-31-33-19:~/( X + v]# docker images
REPOSITORY TAG IMAGE ID CREATED SIZE
cloudfreeks-s3-php latest b31be780b9c5 47 seconds ago 541MB
[ec2-user@ip-172-31-33-19:~/( X + v]#
```


Verification

- The EC2 build machine was launched successfully.
- Docker was installed and works correctly.
- PHP, git, unzip, and Composer were installed.
- The PHP application repository was cloned successfully.
- The required PHP dependency was installed with Composer.
- The Docker image was built successfully with `docker build -t cloudfreeks-s3-php-app`.
- The image appears in docker images.
- The Docker image is ready for the next project step, where it will be tested and then pushed to ECR.